



# BERT

## BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

저자 : Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova

발행일 : 2018년 10월

### 0. Abstract

- BERT(**B**idirectional **E**ncoder **R**epresentations from **T**ransformers)는 말 그대로 양방향 문맥을 활용한 Transformer 기반 언어 모델로, 비지도 학습을 통해 사전 학습된다.
- 사전 학습된 모델에 출력 layer 하나만 추가하면 다양한 NLP 작업에 적용할 수 있으며, 이를 통해 복잡한 아키텍처 수정 없이도 SOTA 성능을 달성하였다.
- GLUE, MultiNLI, SQuAD 등에서 기존 모델보다 뛰어난 성능 향상을 보이며, 효율적인 사전 학습과 미세 조정(fine-tuning)이 가능하다.

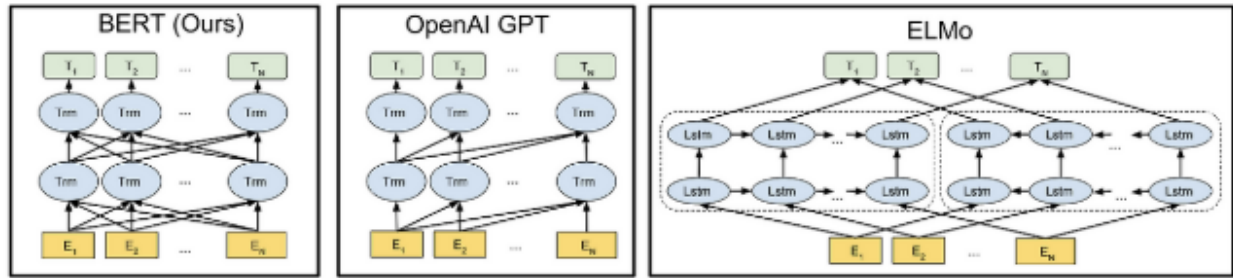
### 1. Introduction

자연어 처리에서 언어 모델의 사전 학습은 효과적이며, token-level뿐만 아니라 QA 같은 sentence-level 작업에도 적용되는 연구가 진행되고 있음

Pre-trained language representation을 downstream task에 적용하는 방식

- Feature-based 접근법 (ex. ELMo)
  - Pre-trained representation을 추가적인 feature로 활용
- Fine-tuning 접근법 (ex. GPT-1)
  - 최소한의 task-specific parameter를 추가하여 효율적인 fine-tuning 수행

## BERT 모델



Differences in pre-training model architectures

- Fine-tuning 기반 접근법을 개선한 모델
- 기존 모델의 Unidirectional 제약을 해결하기 위해 Masked Language Model(MLM) 도입
  - 입력 일부를 마스킹(mask)하고 해당 단어를 문맥을 활용해 예측
  - 양방향(bidirectional) 문맥 학습 가능
- 더 깊은(deep) bidirectional representation 학습 가능
- Next Sentence Prediction(NSP) 추가 → 문장 간 관계 학습 가능

## 2. Related Work

### 2.1. Unsupervised Feature-based Approaches

단어, 문장, 문단 수준에서 사전 학습된 임베딩을 feature로 사용하는 방식

대표적인 모델 : ELMo(Embeddings from Language Model)

- 기존의 word embedding과 달리 문맥(context)에 따라 달라지는 단어 표현을 학습
- 좌우 독립적인 LSTM을 결합하여 문맥 정보를 반영
- 각 층의 가중치를 조절하여 다양한 NLP 작업에 적용 가능

### 2.2. Unsupervised Fine-tuning Approaches

초기 연구들은 단어 임베딩을 비지도 학습 방식으로 사전 학습하는데 집중함

최근에는 문장, 문서 수준의 컨텍스트 임베딩을 사전 학습한 후, supervised downstream tasks에 파인튜닝 하는 방식이 연구되고 있다.

대표적인 모델 : GPT-1

- Transformer의 decoder만 사용 (left-to-right 언어 모델)
- GLUE 벤치마크에서 최고 성능 기록

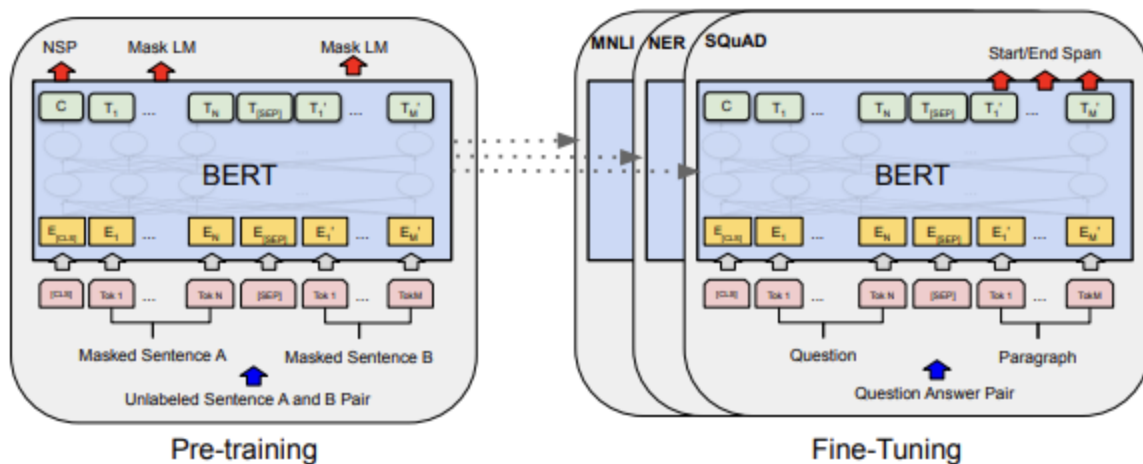
## 2.3. Transfer Learning from Supervised Data

대규모 데이터셋에서 학습된 지도 모델을 다른 작업에 전이하여 활용

대표적으로 자연어 추론 모델, 기계 번역 모델 등에 사용

# 3. BERT

## 3.0. BERT 소개



- BERT의 프레임워크는 두 단계로 구성 : Pre-training / Fine-tuning
- 이때, 사전 학습된 모델의 파라미터를 그대로 초기화한 후, 특정 다운스트림 작업에 맞게 라벨이 있는 데이터로 미세 조정(Fine-tuning)하여 최적화함
- 이 과정에서 모델 구조는 거의 동일하며, 출력층만 작업에 맞게 조정

## Model Architecture

BERT는 Transforemer를 기반으로 한 다층 양방향 Encoder

L : Transformer layer 개수 / H : hidden 크기 / A : Self-attention 헤드 개수

- BERT\_base (L = 12, H = 768, A = 12, Total Parameter = 110M)
- BERT\_large (L = 24, H = 1024, A = 16, Total Parameter = 340M)

→ BERT는 GPT와 거의 유사하지만, decoder가 아닌 encoder를 사용하는 점, unidirectional가 아니라 bidirectional인 점이 다르다.

### Input/Output Representations

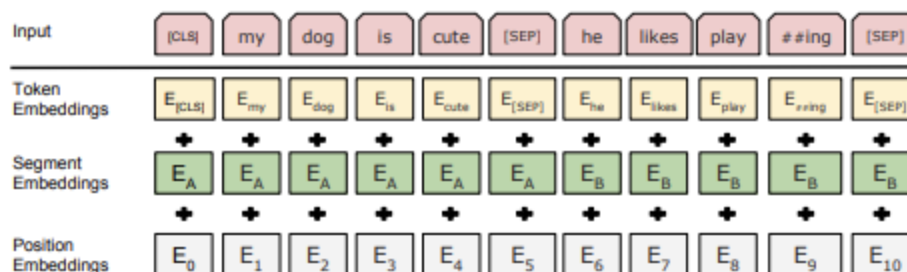
BERT는 다양한 다운스트림 작업에서 single sentence와 sentence pair를 하나의 token sequence로 처리할 수 있도록 설계됨

#### (1) 입력 구성

- WordPiece Embedding 사용 (30,000개 토큰 어휘집)
- 특수 토큰 사용
  - [CLS] → 모든 입력 시퀀스의 첫 번째 토큰 → 최종 출력에서 분류(classification) 작업에 사용됨
  - [SEP] → 두 개의 문장을 구분하는 역할
- 입력 형태
  - 단일 문장 입력: [CLS] + 문장 + [SEP]
  - 문장 쌍 입력 (예: 질문-답변, 자연어 추론): [CLS] + 문장 A + [SEP] + 문장 B + [SEP]

#### (2) 입력 임베딩 구성

BERT의 입력 벡터는 3가지 임베딩 (Token + Segment + Position)을 더하여 생성됨



- Token Embedding → 각 단어의 의미 표현
- **Segment Embedding** → 문장 A와 문장 B를 구분하는 역할 (기존 GPT에는 없음)
- Position Embedding → 단어의 위치 정보를 추가 (Transformer 모델 특성상 필요)

### (3) 출력 처리

- **[CLS]** 토큰의 최종 히든 벡터는 분류(classification) 작업에 사용

## 3.1. Pre-training BERT

BERT는 양방향 언어 모델을 학습하기 위해 두 가지 비지도 학습 방식으로 사전 학습을 진행함

### Task #1: Masked LM

#### (1) 왜 MLM이 필요한가?

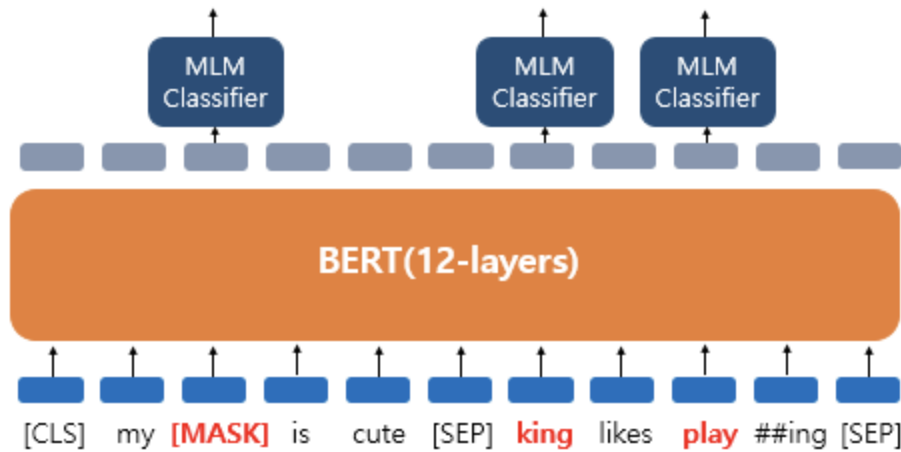
- 기존의 일반적인 언어 모델은 단방향 학습(left-to-right, right-to-left)만 가능하다. 완전한 양방향 학습을 할 경우, 각 단어가 자기 자신을 참조하게 되어 정답을 쉽게 예측할 위험이 있다.
- 완전한 양방향 모델을 학습하기 위해 입력 문장에서 일부 단어를 랜덤하게 마스킹하고, 이를 예측하도록 학습하는 **Masked LM(MLM)** 방식을 사용한다.

#### (2) MLM 방식

- 학습 과정
  - 입력 문장에서 15%의 WordPiece 토큰을 랜덤하게 마스킹
  - 마스킹된 단어의 최종 hidden vector를 softmax를 통해 원래 단어를 예측하도록 학습
  - 일반적인 언어 모델과 동일하게 단어 예측 작업을 수행
- 이는 기존의 Denoising Autoencoder(Vincent et al., 2008)와 다름
  - Autoencoder는 전체 입력을 복원하는 방식이지만, MLM은 마스킹된 단어만 예측하는 방식

#### (3) Pre-training과 Fine-tuning 간의 차이 해결

- 문제 : [MASK] 토큰은 사전 학습(Pre-training) 단계에서만 사용되고, Fine-tuning 시에는 등장하지 않음 → 사전 학습된 모델이 [MASK] 토큰에 과도하게 의존할 수도 있음
- 해결 : [MASK] 토큰을 무조건 사용하는 것이 아니라, 다음과 같은 방식으로 변형하여 학습 진행



출처 : <https://wikidocs.net/115055>

- 먼저, Training data generator가 랜덤으로 15%만의 token position을 선택
  - 만약 i번째 토큰이 선택되었다면, i번째 토큰을 80%의 확률로 [MASK] 토큰으로 대체, 10% 확률로 랜덤한 다른 단어로 변경, 10% 확률로 원래 단어 그대로 유지
  - 그 다음 원래 토큰을 cross entropy loss를 예측하는데 사용
- 이렇게 하면 모델이 [MASK]에 의존하지 않고, 더 일반적인 언어 표현을 학습할 수 있음

## Task #2. Next Sentence Prediction(NSP)

### (1) 왜 NSP가 필요한가?

- QA, NLI 같은 다운스트림 작업에서는 문장 간 관계를 이해하는 것이 필수적
- 하지만 일반적인 언어 모델은 단어 단위의 예측만 수행하므로 문장 간 관계를 직접적으로 학습하지 못함

→ 이를 해결하기 위해 문장 간 관계를 학습할 수 있도록 Next Sentence Prediction(NSP) 과제를 도입

## (2) Next Sentence Prediction(NSP)

- 단일 corpus에서 sentence A, sentence B. 단순히 이 A, B가 다음문장이냐 아니냐 이진 분류(IsNext, NotNext)하는 task를 수행합니다.
- 학습 과정
  - Sentence A, Sentence B를 선정한다.
  - **IsNext**
    - Sentence A: We argue that current techniques restrict the power of the pre-trained representations
    - Sentence B: especially for the fine-tuning approaches.
  - **NotNext**
    - Sentence A: We argue that current techniques restrict the power of the pre-trained representations
    - Sentence B: The major limitation is that standard language models are unidirectional

→ 이는 아주 단순한 방식임에도 불구하고 QA, NLI 같은 문장 관계를 학습해야 하는 작업에서 매우 유용했음

### Pre-training data

- BERT의 사전 학습 과정은 기존 언어 모델 사전 학습 방식과 유사
- 사용된 주요 데이터셋
  - BooksCorpus (8억 개 단어)
  - English Wikipedia (25억 개 단어)

## 3.2. Fine-tuning BERT

### (1) Fine-tuning

- BERT는 Transformer의 Encoder 구조만 사용하여, Self-attention 기법을 활용해 다양한 다운스트림 작업에 쉽게 Fine-tuning 가능

- 기존 연구는 문장 쌍(text pairs)을 독립적으로 인코딩한 후 Cross-attention을 적용했지만,  
BERT는 Self-attention을 통해 이 과정을 통합하여 한 번에 처리 → Bidirectional Cross-attention을 효과적으로 포함할 수 있음

## (2) 과정

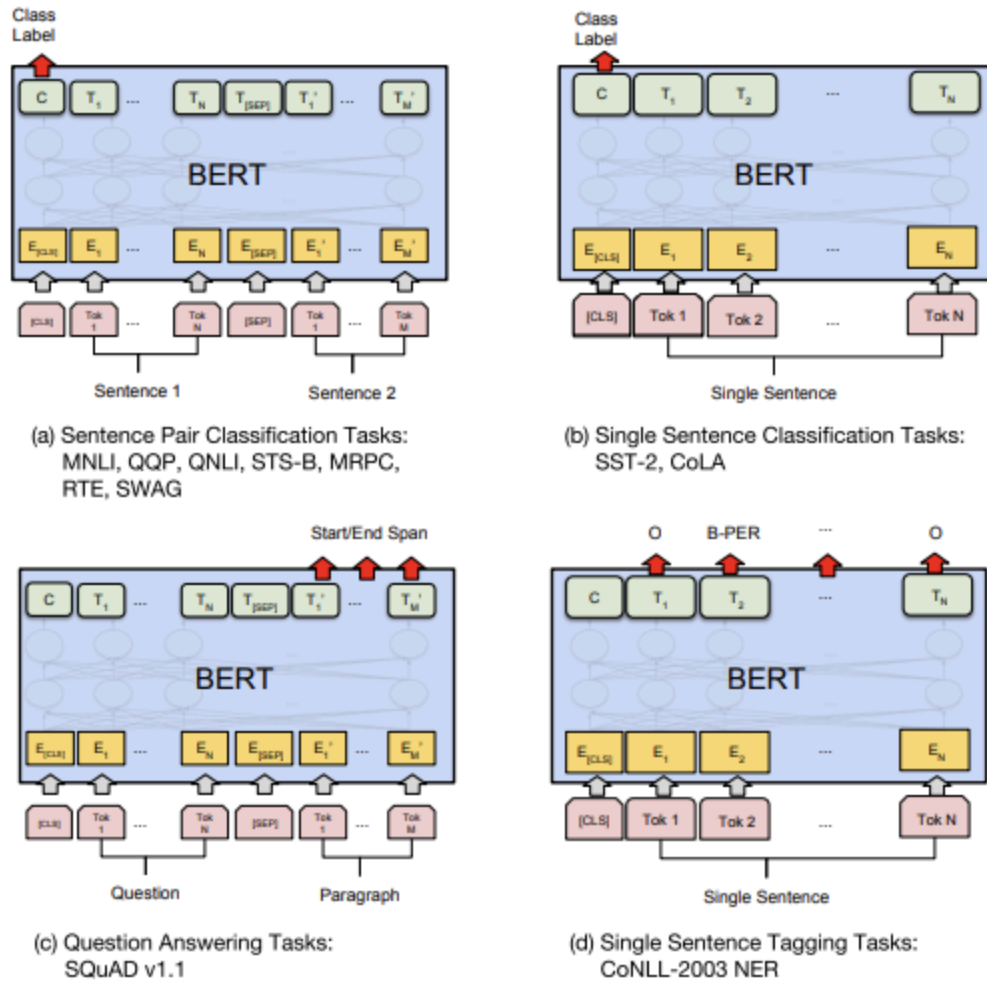


Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

- BERT에 적절한 Input/Output을 연결하고, End-to-End 방식으로 Fine-tuning 진행
- Pre-training에서 학습한 Sentence A, Sentence B 구조를 다운스트림 작업에 맞게 적용
  1. Paraphrasing(문장 유사도 평가) → Sentence A, B = 문장 쌍



2. Entailment(자연어 추론, NLI) → Sentence A, B = 가설(Hypothesis) - 전제(Premise) 쌍
  3. Question Answering(질의응답, QA) → Sentence A, B = 질문(Question) - 문단(Passage) 쌍
- 작업 유형에 따른 Fine-tuning 방식
    - Sequence-level Task (문장 단위)
    - Token-level Task (토큰 단위)

## 4. Experiments

### 4.1. GLUE

GLUE (General Language Understanding Evaluation) 벤치마크는 다양한 자연어 이해 (NLU) 작업을 평가하는 데이터셋 모음

- 단일 문장(Single Sentence) 또는 문장 쌍(Sentence Pair)을 Section 3에서 설명한 방식으로 입력
- [CLS] 토큰의 최종 히든 벡터(C)를 전체 입력의 표현으로 사용
- Fine-tuning 과정에서 추가되는 유일한 새로운 파라미터 = 분류 레이어의 가중치 W

| System                | MNLI-(m/mm)<br>392k | QQP<br>363k | QNLI<br>108k | SST-2<br>67k | CoLA<br>8.5k | STS-B<br>5.7k | MRPC<br>3.5k | RTE<br>2.5k | Average<br>- |
|-----------------------|---------------------|-------------|--------------|--------------|--------------|---------------|--------------|-------------|--------------|
| Pre-OpenAI SOTA       | 80.6/80.1           | 66.1        | 82.3         | 93.2         | 35.0         | 81.0          | 86.0         | 61.7        | 74.0         |
| BiLSTM+ELMo+Attn      | 76.4/76.1           | 64.8        | 79.8         | 90.4         | 36.0         | 73.3          | 84.9         | 56.8        | 71.0         |
| OpenAI GPT            | 82.1/81.4           | 70.3        | 87.4         | 91.3         | 45.4         | 80.0          | 82.3         | 56.0        | 75.1         |
| BERT <sub>BASE</sub>  | 84.6/83.4           | 71.2        | 90.5         | 93.5         | 52.1         | 85.8          | 88.9         | 66.4        | 79.6         |
| BERT <sub>LARGE</sub> | <b>86.7/85.9</b>    | <b>72.1</b> | <b>92.7</b>  | <b>94.9</b>  | <b>60.5</b>  | <b>86.5</b>   | <b>89.3</b>  | <b>70.1</b> | <b>82.1</b>  |

→ 비슷한 모델 사이즈를 가진 OpenAI GPT 대비 벤치마크 점수가 평균적으로 높게 측정

→ BERT Large 모델에서 BERT Base 모델 대비 아주 적은 데이터셋 크기에서 상당히 뛰어난 성능

### 4.2. SQuAD v1.1

100,000개의 질문-답변 데이터셋으로, 주어진 질문과 위키피디아 문단을 입력으로 받아, 정답이 포함된 문장 내에서 정답을 예측하는 작업

- 입력 방식: 질문(Question) = A 임베딩, 문단(Passage) = B 임베딩으로 설정
- Start 벡터(S)와 End 벡터(E)를 추가하여 정답의 시작과 끝 위치를 예측
- 예측 과정
  - 각 토큰 벡터(Ti)와 Start 벡터(S) 사이의 내적(dot product) 후 Softmax 적용 → 정답의 시작 위치 확률 계산
  - 각 토큰 벡터(Ti)와 End 벡터(E) 사이의 내적(dot product) 후 Softmax 적용 → 정답의 끝 위치 확률 계산
  - 최종 정답 점수 계산
  - 가장 높은 점수를 가지는 span이 정답으로 선택됨

| System                                   | Dev         |             | Test        |             |
|------------------------------------------|-------------|-------------|-------------|-------------|
|                                          | EM          | F1          | EM          | F1          |
| Top Leaderboard Systems (Dec 10th, 2018) |             |             |             |             |
| Human                                    | -           | -           | 82.3        | 91.2        |
| #1 Ensemble - nlnet                      | -           | -           | 86.0        | 91.7        |
| #2 Ensemble - QANet                      | -           | -           | 84.5        | 90.5        |
| Published                                |             |             |             |             |
| BiDAF+ELMo (Single)                      | -           | 85.6        | -           | 85.8        |
| R.M. Reader (Ensemble)                   | 81.2        | 87.9        | 82.3        | 88.5        |
| Ours                                     |             |             |             |             |
| BERT <sub>BASE</sub> (Single)            | 80.8        | 88.5        | -           | -           |
| BERT <sub>LARGE</sub> (Single)           | 84.1        | 90.9        | -           | -           |
| BERT <sub>LARGE</sub> (Ensemble)         | 85.8        | 91.8        | -           | -           |
| BERT <sub>LARGE</sub> (Sgl.+TriviaQA)    | <b>84.2</b> | <b>91.1</b> | <b>85.1</b> | <b>91.8</b> |
| BERT <sub>LARGE</sub> (Ens.+TriviaQA)    | <b>86.2</b> | <b>92.2</b> | <b>87.4</b> | <b>93.2</b> |

→ BERT는 기존 최고 모델을 큰 폭으로 능가

→ 단일 모델(single BERT)이 기존의 앙상블 모델보다 더 높은 F1 점수 기록

→ TriviaQA 데이터셋(Joshi et al., 2017)을 추가적으로 활용하여 성능을 향상시킴

### 4.3. SQuAD v2.0

SQuAD v2.0은 SQuAD v1.1의 확장 버전으로, 정답이 존재하지 않는(No Answer) 경우도 포함됨

- 정답이 없는 경우를 처리하기 위해 [CLS] 토큰을 "No Answer"로 활용

- 정답이 있는 경우와 없는 경우를 비교하여 정답이 있는 경우만 예측하도록 설계

| System                                   | Dev  |      | Test |      |
|------------------------------------------|------|------|------|------|
|                                          | EM   | F1   | EM   | F1   |
| Top Leaderboard Systems (Dec 10th, 2018) |      |      |      |      |
| Human                                    | 86.3 | 89.0 | 86.9 | 89.5 |
| #1 Single - MIR-MRC (F-Net)              | -    | -    | 74.8 | 78.0 |
| #2 Single - nlnet                        | -    | -    | 74.2 | 77.1 |
| Published                                |      |      |      |      |
| unet (Ensemble)                          | -    | -    | 71.4 | 74.9 |
| SLQA+ (Single)                           | -    | -    | 71.4 | 74.4 |
| Ours                                     |      |      |      |      |
| BERT <sub>LARGE</sub> (Single)           | 78.7 | 81.9 | 80.0 | 83.1 |

→ BERT는 기존 최고 모델을 +5.1 F1 향상시킴

→ TriviaQA 데이터를 추가하지 않고도 기존 모델을 능가하는 성능 기록

#### 4.4. SWAG

SWAG (Situations With Adversarial Generations) 데이터셋은 113,000개의 문장 쌍 (sentence-pair) 완성 문제로 구성됨

주어진 문장(Sentence A)에 대해 가장 적절한 다음 문장(Sentence B)을 4가지 선택지 중에서 고르는 작업

- 각 문장 쌍(문장 A + 4가지 문장 B 후보)을 개별적으로 입력하여 총 4개의 입력 시퀀스를 생성
- [CLS] 토큰의 최종 벡터(**c**)와 새로운 벡터를 내적(dot product)하여 각 선택지에 대한 점수를 계산
- Softmax를 적용하여 가장 높은 점수를 받은 선택지를 정답으로 예측

| System                             | Dev         | Test        |
|------------------------------------|-------------|-------------|
| ESIM+GloVe                         | 51.9        | 52.7        |
| ESIM+ELMo                          | 59.1        | 59.2        |
| OpenAI GPT                         | -           | 78.0        |
| BERT <sub>BASE</sub>               | 81.6        | -           |
| BERT <sub>LARGE</sub>              | <b>86.6</b> | <b>86.3</b> |
| Human (expert) <sup>†</sup>        | -           | 85.0        |
| Human (5 annotations) <sup>†</sup> | -           | 88.0        |

→ BERT<sub>LARGE</sub>는 기존 최고 모델을 크게 능가

→ ESIM+ELMo 시스템 대비 +27.1% 성능 향상

→ OpenAI GPT 대비 +8.3% 성능 향상

## 5. Ablation Studies

### 5.1. Effect of Pre-training Tasks

두 가지 핵심 기법(MLM과 NSP)이 모델 성능에 미치는 영향을 분석하기 위해 제거(Ablation) 실험을 수행

- No NSP 모델
  - MLM(Masked Language Model)만 사용하고, NSP(Next Sentence Prediction) 없이 학습
  - 즉, 문장 간 관계 학습을 제외한 버전
- LTR & No NSP 모델
  - 기존의 Left-to-Right(LTR) 언어 모델을 사용하고, NSP 없이 학습
  - 즉, 단방향(unidirectional) 학습을 수행하며, OpenAI GPT와 유사한 방식

| Tasks                | Dev Set         |               |               |                |               |
|----------------------|-----------------|---------------|---------------|----------------|---------------|
|                      | MNLI-m<br>(Acc) | QNLI<br>(Acc) | MRPC<br>(Acc) | SST-2<br>(Acc) | SQuAD<br>(F1) |
| BERT <sub>BASE</sub> | 84.4            | 88.4          | 86.7          | 92.7           | 88.5          |
| No NSP               | 83.9            | 84.9          | 86.5          | 92.6           | 87.9          |
| LTR & No NSP         | 82.1            | 84.3          | 77.5          | 92.1           | 77.8          |
| + BiLSTM             | 82.1            | 84.1          | 75.7          | 91.6           | 84.9          |

→ NSP를 제거하면(QNLI, MNLI, SQuAD 1.1)에서 성능이 크게 하락. 이는 문장 간 관계를 이해하는 능력이 중요한 다운스트림 작업에서 NSP가 필수적임을 의미

→ LTR 모델이 모든 작업에서 MLM 모델보다 성능이 낮으며 특히 MRPC와 SQuAD 작업에서 성능 감소가 두드러짐

## 5.2. Effect of Model Size

BERT 모델의 크기가 Fine-tuning 성능에 미치는 영향을 분석하기 위해 다양한 크기의 모델을 학습

레이어 수(L), 히든 유닛 크기(H), 어텐션 헤드 개수(A)를 변경하면서 실험 진행

| Hyperparams |      |    |          | Dev Set Accuracy |      |       |
|-------------|------|----|----------|------------------|------|-------|
| #L          | #H   | #A | LM (ppl) | MNLI-m           | MRPC | SST-2 |
| 3           | 768  | 12 | 5.84     | 77.9             | 79.8 | 88.4  |
| 6           | 768  | 3  | 5.24     | 80.6             | 82.2 | 90.7  |
| 6           | 768  | 12 | 4.68     | 81.9             | 84.8 | 91.3  |
| 12          | 768  | 12 | 3.99     | 84.4             | 86.7 | 92.9  |
| 12          | 1024 | 16 | 3.54     | 85.7             | 86.9 | 93.3  |
| 24          | 1024 | 16 | 3.23     | 86.6             | 87.8 | 93.7  |

→ 모델 크기가 커질수록 모든 GLUE 작업에서 성능이 향상

→ MRPC(문장 유사도)처럼 훈련 데이터가 적은 작업에서도 큰 모델이 더 높은 성능을 보임

→ BERTLARGE가 BERTBASE보다 확연히 우수한 성능을 기록

## 5.3. Feature-based Approach with BERT

BERT는 Fine-tuning 방식뿐만 아니라 Feature-based 방식으로도 효과적으로 활용 가능

| System                                         | Dev F1 | Test F1     |
|------------------------------------------------|--------|-------------|
| ELMo (Peters et al., 2018a)                    | 95.7   | 92.2        |
| CVT (Clark et al., 2018)                       | -      | 92.6        |
| CSE (Akbik et al., 2018)                       | -      | <b>93.1</b> |
| Fine-tuning approach                           |        |             |
| BERT <sub>LARGE</sub>                          | 96.6   | 92.8        |
| BERT <sub>BASE</sub>                           | 96.4   | 92.4        |
| Feature-based approach (BERT <sub>BASE</sub> ) |        |             |
| Embeddings                                     | 91.0   | -           |
| Second-to-Last Hidden                          | 95.6   | -           |
| Last Hidden                                    | 94.9   | -           |
| Weighted Sum Last Four Hidden                  | 95.9   | -           |
| Concat Last Four Hidden                        | 96.1   | -           |
| Weighted Sum All 12 Layers                     | 95.5   | -           |

→ NER 실험 결과, Fine-tuning이 가장 높은 성능을 보였으나, Feature-based 방식도 강력한 성능을 기록

→ Feature-based 방식에서 "마지막 4개 은닉층을 결합한 표현(Concat Last Four Hidden)"이 Fine-tuning과 가장 가까운 성능을 보임

## 6. Conclusion

- BERT는 양방향 컨텍스트 정보를 활용한 사전 학습을 통해 강력한 전이 학습(Transfer Learning) 성능을 입증
- 다양한 다운스트림 작업에서 높은 성능을 보이며, 기존 단방향 모델보다 더 효과적
- 적은 양의 라벨 데이터(low-resource tasks)만으로도 높은 정확도를 달성할 수 있음
- 사전 학습된 하나의 모델을 다양한 NLP 작업에 적용할 수 있어 확장성과 활용성이 뛰어남